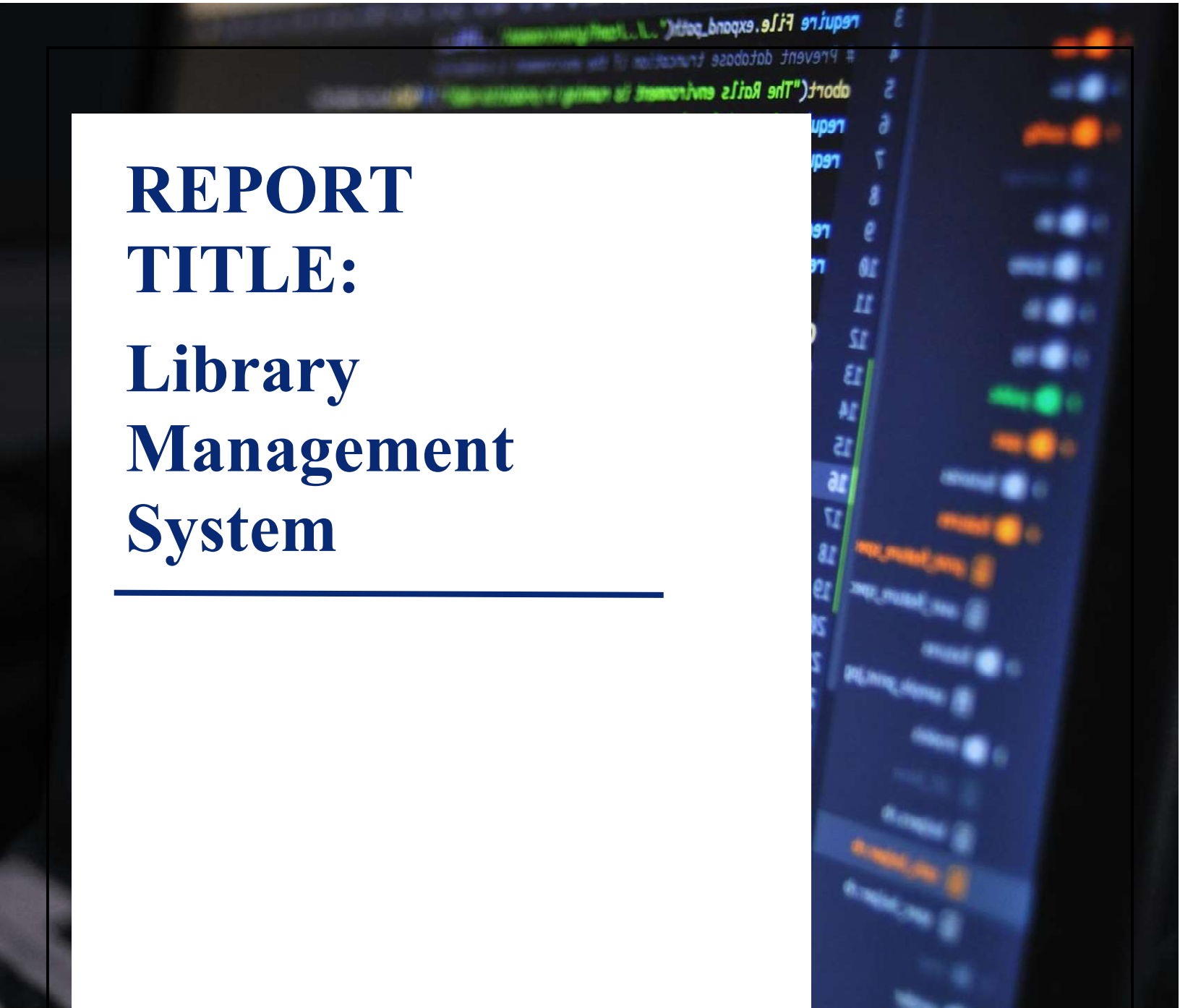




REPORT TITLE: Library Management System

Group Members:-

Areeba Hameed (2023-BCS-015)

Irza Hameed (2023-BCS-044)

Isra Asif (2023-BCS-045)

Maria Yasmeen (2023-BCS-059)

Theory Instructor: Ma'am Nausheen Saba

Lab Instructor: Sir Adeel Khalid

Basics Components

- Swing packages
- AWT packages
- JButton
- JLabel
- JTextfield
- JTextArea
- JPanel

Basics Concept

- Inheritance
- Abstraction
- Encapsulation
- Polymorphism
- Event Handling
- Composition

Code Snippet:-

```
package com.hr.test;

import javax.swing.JFrame;
import javax.swing.JPanel;
import javax.swing.border.EmptyBorder;
import javax.swing.JLabel;
import javax.swing.ImageIcon;
import javax.swing.JButton;
import java.awt.Color;
import java.awt.Font;
import java.awt.event.ActionListener;
import java.awt.event.ActionEvent;

public class accounts extends JFrame {
```

```
private static final long serialVersionUID = 1L;
private JPanel contentPane;

public accounts() {
    setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    setBounds(100,100,965,550);
    contentPane = new JPanel();
    contentPane.setBorder(new EmptyBorder(5, 5, 5, 5));

    setContentPane(contentPane);
    contentPane.setLayout(null);

    JButton Update = new JButton("Update");
    Update.addActionListener(new ActionListener() {
        public void actionPerformed(ActionEvent e) {
            update_accounts frame = new update_accounts();
            frame.setVisible(true);
            dispose();
        }
    });
    Update.setForeground(Color.WHITE);
    Update.setFont(new Font("Arial Black", Font.BOLD, 15));
    Update.setBackground(new Color(10, 99, 119));
    Update.setBounds(388, 267, 193, 51);
    contentPane.add(Update);

    JButton Delete = new JButton("Delete");
    Delete.addActionListener(new ActionListener() {
        public void actionPerformed(ActionEvent e) {
            delete_accounts frame = new delete_accounts();
            frame.setVisible(true);
            dispose();
        }
    });
    Delete.setForeground(Color.WHITE);
```

```
Delete.setFont(new Font("Arial Black", Font.BOLD, 15));
Delete.setBackground(new Color(10, 99, 119));
Delete.setBounds(388, 206, 193, 51);
contentPane.add>Delete);
```

```
JLabel lblNewLabel_1 = new JLabel("ACCOUNTS");
lblNewLabel_1.setForeground(new Color(0, 0, 0));
lblNewLabel_1.setFont(new Font("Times New Roman", Font.BOLD, 70));
lblNewLabel_1.setBounds(255, 10, 418, 147);
contentPane.add(lblNewLabel_1);
```

```
JButton Add = new JButton("Add");
Add.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent e) {
        add_accounts frame = new add_accounts();
        frame.setVisible(true);
        dispose();
    }
});
```

```
Add.setForeground(Color.WHITE);
Add.setFont(new Font("Arial Black", Font.BOLD, 15));
Add.setBackground(new Color(10, 99, 119));
Add.setBounds(388, 143, 193, 51);
contentPane.add>Add);
```

```
JButton Back = new JButton("Back");
Back.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent e) {
        librarian frame = new librarian(null);
        frame.setVisible(true);
dispose();
    }
});
```

```
Back.setForeground(Color.WHITE);
Back.setFont(new Font("Arial Black", Font.BOLD, 15));
Back.setBackground(new Color(10, 99, 118));
```

```
        Back.setBounds(103, 396, 193, 51);
        contentPane.add(Back);

        JLabel lblNewLabel = new JLabel("");
        lblNewLabel.setIcon(new
ImageIcon("C:\\Users\\Name\\OneDrive\\Desktop\\9.jpg"));
        lblNewLabel.setBounds(-200, -33, 1183, 546);
        contentPane.add(lblNewLabel);
    }
}
```

Components Explanation:-

The code above defines a **Java Swing GUI application** for managing accounts. Let's break down the key concepts used in this code:

1. JFrame and JPanel:

- The `accounts` class extends `JFrame`, which represents a top-level window or frame in a Swing application.
- The `contentPane` is a `JPanel` (a lightweight container) that holds other components within the frame.

2. Buttons:

- Three buttons are defined: "Add," "Update," and "Delete."
- Each button has an associated action listener that opens a new frame (e.g., `add\_accounts`, `update\_accounts`, or `delete\_accounts`) when clicked.

3. Labels:

- The label "ACCOUNTS" is displayed in large font using the `JLabel` component.
- It uses the "Times New Roman" font with a size of 70.

4. Background Image:

- The background image is set using the `ImageIcon` class.

- The image file path is specified as `"C:\\Users\\Name\\OneDrive\\Desktop\\9.jpg"` (you should replace this with the actual path to your image).

5. Button Actions:

- When the "Add" button is clicked, it opens the ``add_accounts`` frame.
- When the "Update" button is clicked, it opens the ``update_accounts`` frame.
- When the "Delete" button is clicked, it opens the ``delete_accounts`` frame.
- The "Back" button returns to the ``librarian`` frame.

6. Window Size and Layout:

- The frame size is set to 965x550 pixels.
- Components are positioned using absolute layout (``setLayout(null)``)

Java Concept Explanation:-

The Java concepts used in the provided code snippet are listed below

1. Inheritance:

The accounts class extends the JFrame class. This demonstrates inheritance, where accounts inherits properties and behaviors (methods) from the JFrame class.

Inheritance allows you to create a new class based on an existing class, reusing its functionality.

2. Abstraction:

The accounts class itself serves as an abstraction. It encapsulates the details of creating a frame, buttons, labels, and their associated actions.

The user of this class doesn't need to know the low-level implementation details; they interact with the high-level interface provided by the class.

3. Encapsulation:

The class encapsulates the GUI components (buttons, labels) and their behavior within a single unit (accounts).

The private fields, methods, and their interactions are encapsulated within the class.

4. Polymorphism:

Polymorphism is not explicitly demonstrated in this code snippet.

However, if you were to add more buttons (e.g., “View,” “Search”), each with its own action, you’d have a form of polymorphism (method overloading or method overriding).

5. Event Handling:

The action listeners for buttons (Update, Delete, Add, Back) demonstrate event handling.

When a button is clicked, the associated action (opening a new frame) is triggered.

6. Composition:

The accounts class composes its GUI components (buttons, labels) by adding them to the contentPane.

Composition allows building complex objects by combining simpler objects.

Benefits:-

The benefits of using components and Java concepts in the provided code snippet are listed below:

1. Components (Buttons, Labels, Panels):

User Interaction: Components like buttons (Add, Update, Delete, Back) allow users to interact with the application. They trigger specific actions when clicked.

Visual Presentation: Labels (ACCOUNTS) provide visual information to users, enhancing the user interface.

Layout Management: Panels (contentPane) help organize and position components within the frame.

2. Inheritance:

Code Reusability: By extending JFrame, the accounts class inherits built-in features for creating and managing frames. This reduces redundant code and promotes reusability.

3. Abstraction:

High-Level Interface: The accounts class abstracts away low-level details of frame creation. Users interact with the high-level interface (buttons, labels) without needing to know the implementation specifics.

4. Encapsulation:

Data Hiding: Encapsulation hides the internal details of components. Users don't need to know how buttons or labels are implemented; they only use their functionality.

Modularity: Each component (button, label) is encapsulated within the accounts class, promoting modular design.

5. Event Handling:

Responsive Behavior: Event listeners (e.g., button actions) handle user interactions. When a button is clicked, the associated action (opening a new frame) is executed.

Decoupling: Event handling decouples user input from application logic.

6. Composition:

Component Composition: The accounts class composes its GUI components (buttons, labels) to create a cohesive user interface.

Complexity Management: Composition allows building complex objects by combining simpler ones.

7. Customization and Flexibility:

Custom UI: Using components allows customization of appearance (font, color, layout) to match the application's design.

Dynamic Behavior: Buttons trigger different actions (add, update, delete), providing flexibility based on user needs.

8. Readability and Maintainability:

Structured Code: Components and their actions are organized within the class. This improves code readability and maintainability.

Easy Debugging: Isolating components simplifies debugging and troubleshooting.

9. Polymorphism (Potential):

While not explicitly demonstrated here, adding more buttons (e.g., "View," "Search") would showcase polymorphism (method overloading or overriding).

10. GUI Development Efficiency:

Swing Framework: Leveraging Swing components accelerates GUI development compared to manual UI creation.

Consistent Look and Feel: Swing components provide a consistent look across platforms.

In conclusion, using components and Java concepts enhances code organization, user experience, and maintainability. It promotes best practices in software development.
